BrightLearn

MACHINE LEARNING CASE STUDY · STUDENT BRIEF

Predicting 30-Day Patient Readmission

Build, evaluate and explain a predictive model in Python for a healthcare startup, working through data preprocessing, exploratory analysis, feature selection, model building and evaluation.

Python | Pandas & NumPy | Matplotlib & Seaborn | Scikit-learn

Formative Assessment · 100 marks

2026

The case study at a glance

LEVEL	MODULE	ASSESSMENT	MARKS	SUBMIT
Postgraduate	Programming (Python)	Formative · Project	100	Report + notebook

A healthcare startup wants to know which patients are likely to return to hospital within 30 days of being discharged. You have been given their historical patient data. Your job is to clean it, explore it, choose the right predictors, build and tune a classification model in Python, and evaluate how well it works, then write it all up as a professional report that a data science lead and a hospital manager could both read.

Contents

The scenario and your task	What the healthcare startup needs from you	3
The dataset	Fields, units, a preview and how to load it	4
The five tasks	What to do and what to show, task by task	6
Task 1 · Data preprocessing	20 marks	7
Task 2 · Exploratory data analysis	20 marks	8
Task 3 · Feature selection	20 marks	9
Task 4 · Model building	20 marks	10
Task 5 · Model evaluation and testing	10 marks	11
Report format and marking	Required structure and the marking scheme	12
Submission, workflow and integrity	What to hand in, a suggested timeline, academic honesty	13
Appendix A · Python quick reference	The functions you are likely to need for each task	14
Appendix B · Report template and checklist	Headings to use and a self-check before you submit	16

How to use this brief

Read the scenario and the dataset section first, then work through the five tasks in order: each one depends on the one before it. Every task page tells you **what to do**, **what to show in your report** and the **questions a marker will ask**. The report format and marking scheme on page 12 tell you how the 100 marks are allocated. Appendix A lists the Python functions you will need, and Appendix B gives you a report template and a checklist to run through before you submit.

THE SCENARIO

The scenario and your task

What the healthcare startup needs from you

Scenario

A healthcare startup is interested in developing a machine learning model to predict patient readmission based on various patient data points collected from their hospital visits. The company aims to use Python's extensive libraries and machine learning frameworks to analyse the provided historical patient data and build a predictive model that can help hospitals reduce unnecessary readmissions, saving costs and improving patient care.

Unplanned readmissions within 30 days of discharge are one of the most closely watched measures of hospital quality: expensive for hospitals and funders, disruptive for patients, and often avoidable with timely follow-up. The difficulty is knowing which patients need that attention. Published readmission models achieve only modest accuracy (Kansagara et al., 2011), and simple, explainable scores such as the LACE index remain in wide use (van Walraven et al., 2010). The startup therefore wants a model that is as accurate as the data allow, **understandable** and **honestly evaluated**.

Your task

MAIN QUESTION

Develop a predictive model using Python to assess the likelihood of patient readmission within 30 days of discharge. Your work should use Python's data science libraries to process and analyse the data, build a predictive model, and evaluate its effectiveness.

The project is divided into five tasks that mirror the stages of a real machine learning project (90 marks); the remaining 10 marks reward the quality of the report itself.

1	2	3	4	5
Data preprocessing	**Exploratory analysis**	**Feature selection**	**Model building**	**Model evaluation**
20 marks	20 marks	20 marks	20 marks	10 marks

What the startup expects to receive

- ✓ A **written report** (structure on page **12**) for a technical lead who will check your method and a hospital manager who will act on your findings.
- ✓ A **Jupyter notebook** (or Python script) with all the code, runnable from top to bottom on the supplied dataset, with outputs visible.
- ✓ A **clear recommendation**: which model, which features, how well it performs on unseen data, and how a hospital should use it.

THE DATASET

The dataset

One row per hospital admission

FILE	RECORDS	FIELDS	TARGET
`Expanded_Patient_Readmission_Data_1.csv`	3,000 admissions	10 columns	Readmission (Yes / No)

Data dictionary

Each row describes one hospital admission and whether that patient was readmitted within 30 days. The fields are listed below with their values exactly as they appear in the file. Column names contain spaces and capital letters; you may rename them in your code, but say so in your report.

FIELD	TYPE	DESCRIPTION	VALUES AS RECORDED
Patient ID	Integer	Identifier for the admission record	1 – 3000
Age	Numeric	Age of the patient at admission, in years	18 – 89
Gender	Categorical	Gender as recorded	Female / Male
Admission Type	Categorical	Whether the admission was planned (elective) or unplanned (emergency)	Elective / Emergency
Length of Stay	Numeric	Days between admission and discharge	1 – 29 days
Number of Diagnoses	Numeric	Number of diagnoses recorded during the stay	1 – 9
Blood Pressure	Numeric	Blood pressure reading at admission (mmHg)	60 – 179
Blood Sugar Levels	Numeric	Blood sugar reading at admission (mg/dL)	51 – 199
Previous Admissions	Numeric	Number of earlier hospital admissions on record	0 – 4
Readmission	**Target**	Whether the patient was readmitted within 30 days of discharge	Yes / No

The first five rows

PATIENT ID	AGE	GENDER	ADMISSION TYPE	LENGTH OF STAY	NUMBER OF DIAGNOSES	BLOOD PRESSURE	BLOOD SUGAR LEVELS	PREVIOUS ADMISSIONS	READMISSION
1	62	Female	Elective	4	5	110	130	1	No
2	65	Male	Emergency	19	2	157	81	4	No
3	82	Female	Emergency	18	4	74	84	0	No
4	85	Male	Emergency	2	4	106	85	4	No
5	85	Female	Elective	19	3	80	119	3	No

Loading the data

PYTHON

```python
import pandas as pd

df = pd.read_csv("Expanded_Patient_Readmission_Data_1.csv")
print(df.shape)          # rows, columns
print(df.dtypes)         # data type of each column
df.head()                # first five rows
```

OUTPUT

```
(3000, 10)
```

The data is supplied as exported

Do not assume the file is clean because it loads without errors. Part of Task 1 is to verify its quality: types, ranges, duplicates, missing values and consistency of labels. Report what you find, including when you find nothing wrong; evidence of a careful audit earns marks whatever its outcome.

Treat it as real patient data

The dataset has been prepared for teaching and contains no identifiable individuals. Work with it as you would with real clinical data: do not share it outside the course, do not attempt to link it to any other data, and discuss the ethical implications of a readmission model in your report.

The five tasks

The tasks below are the heart of the assessment and carry 90 of the 100 marks. Work through them in order and keep notes as you go: the decisions you make in Task 1 shape everything that follows, and markers reward decisions that are explained, not just executed.

TASK	FOCUS	MARKS	LIBRARIES NAMED IN THE BRIEF	CORE DELIVERABLE
1	Data preprocessing and cleanup	20	pandas, NumPy	A documented, repeatable cleaning pipeline and a model-ready dataset
2	Exploratory data analysis	20	Matplotlib, Seaborn	Interpreted visualisations of key variables and their relationships
3	Feature selection	20	pandas, SciPy, scikit-learn	A justified final set of predictors
4	Model building	20	scikit-learn	Compared, tuned candidate models and a reasoned selection
5	Model evaluation and testing	10	scikit-learn	Test-set metrics, overfitting analysis and limitations

Three rules that apply to every task

1

Justify every decision

Why the median and not the mean? Why one-hot and not label encoding? Why this threshold? One sentence of reasoning is worth more than a page of code.

2

Never let the test set leak

Split the data before you fit any scaler, imputer or model. Statistics learned from the test rows make your results look better than they are.

3

Interpret, do not just display

Every chart, table and metric must be followed by what it means for the hospital. An uninterpreted figure earns very little.

Keep the audience in mind

Every chart, table and model choice should answer a question the hospital would ask. "Which patients should our nurses phone first?" is a better guiding question than "which algorithm has the highest score?"

ASSESSMENT TIP · **A note on results**

You are marked on the quality of your method, evidence and reasoning, not on reaching a particular score. A carefully built model that turns out to have weak predictive power, reported honestly with a discussion of why, is a strong submission. A model with an impressive score and no explanation of how it was validated is a weak one.

TASK 1 · 20 MARKS

Data preprocessing and cleanup

Turn the raw export into model-ready data, and prove it

20 marks

Task 1 of 5

Aim. Audit the dataset, fix what needs fixing with a defensible rule, encode and scale it for modelling, and do all of this in a way that could be rerun on next month's export.

What to do

1. **Load and inspect.** Report the shape, the data types, the first rows and summary statistics (`describe()`) for numeric and categorical columns.
2. **Audit quality.** Count missing values per column, duplicate rows and duplicate patient IDs. List the unique values of each categorical column and check for inconsistent spelling, case or spacing. Check that numeric values fall within plausible ranges (an age of 0 or a stay of 400 days would need explaining).
3. **Handle missing and invalid values.** Decide, column by column, whether to impute (median, mode, KNN), flag or remove, and justify each choice. If a column has no missing values, say so with evidence and state how you would handle them if they appeared.
4. **Handle outliers.** Identify them with box plots or the IQR rule, decide whether they are errors or real extremes, and treat them accordingly. Removing them without a reason is not acceptable.
5. **Encode categorical variables.** Convert Gender, Admission Type and the target to numeric form (binary or one-hot encoding) and explain why you chose that encoding.
6. **Normalise numeric variables.** Standardise or min-max scale the numeric features, explain why some algorithms need it and others do not, and fit the scaler on training data only.
7. **Split the data.** Create a stratified train/test split (for example 80/20) *before* fitting any imputer or scaler, ideally by placing preprocessing in a scikit-learn `Pipeline` or `ColumnTransformer` .

SHOW IN YOUR REPORT

- ✓ A before-and-after summary table: rows, columns, missing values, duplicates
- ✓ The audit output (missing counts, unique labels, value ranges)
- ✓ A table of every cleaning rule with its justification
- ✓ The encoding and scaling choices, with reasons
- ✓ The train/test split sizes and class balance in each

QUESTIONS A MARKER WILL ASK

- Did you check the data, or assume it was clean?
- Is every change to the data explained?
- Could someone rerun your cleaning on a new file?
- Did any information from the test set reach the training process?
- Why did you scale, and why that scaler?

Common pitfalls

- × Deleting every row with a missing value without checking how many you lose
- × Scaling or imputing the full dataset and splitting afterwards (data leakage)
- × Label-encoding a category as 0/1/2 for a linear model, which implies an order that does not exist
- × Describing the code instead of the decisions

Useful tools

`df.info()` , `df.describe(include="all")` , `df.isna().sum()` , `df.duplicated()` , `df[col].unique()` , `str.strip()` , `pd.get_dummies()` , `SimpleImputer` , `KNNImputer` , `StandardScaler` , `MinMaxScaler` , `OneHotEncoder` , `ColumnTransformer` , `Pipeline` , `train_test_split(stratify=y)`

TASK 2 · 20 MARKS

Exploratory data analysis

Understand the data before you model it

20 marks

Task 2 of 5

Aim. Describe the distribution of every important variable, show how the predictors relate to readmission and to each other, and turn what you see into insights that guide the rest of the project.

What to do

1. **Target balance.** Show how many patients were readmitted and the proportion. Explain what the balance means for the metrics you will use later.
2. **Univariate analysis.** Plot the distribution of each numeric variable (histograms or box plots) and the frequency of each categorical variable (bar charts). Comment on shape, skew, spread and anything unusual.
3. **Bivariate analysis against the target.** Compare readmitted and non-readmitted patients: readmission rate by Gender and Admission Type; box plots or violin plots of Age, Length of Stay, Blood Pressure, Blood Sugar Levels by outcome; readmission rate by Number of Diagnoses and Previous Admissions.
4. **Relationships between predictors.** Produce a correlation matrix (Seaborn heatmap) and, where useful, scatter plots or a pair plot. Identify any pairs of features that carry the same information.
5. **Interpret.** Write one or two sentences under every figure saying what it shows and why it matters. Finish with a short list of the insights you will carry into feature selection and modelling.

SHOW IN YOUR REPORT

- ✓ At least five well-labelled figures made with Matplotlib and Seaborn, numbered and captioned
- ✓ A descriptive statistics table for the numeric variables
- ✓ A sentence of interpretation under every figure
- ✓ A summary of insights at the end of the section

QUESTIONS A MARKER WILL ASK

- Does every chart have a title, axis labels and units?
- Did you choose the right chart for each question (see the BrightLearn *Choosing the Right Chart* guide)?
- Which variables appear to be associated with readmission, and how strongly?
- Did you spot the class imbalance and say what it implies?
- What did you learn that changed your plan?

Common pitfalls

- × Twenty unlabelled charts and no words: volume is not analysis
- × A pie chart for a nine-category variable
- × Reporting a correlation of 0.03 as a "relationship"
- × Treating Patient ID as a variable worth plotting
- × Insights that do not reappear anywhere in Tasks 3 to 5

Useful tools

`df.describe()`, `value_counts(normalize=True)`, `groupby(col)[target].mean()`, `plt.subplots()`, `ax.hist()`, `ax.bar()`, `sns.histplot`, `sns.boxplot`, `sns.violinplot`, `sns.countplot`, `sns.barplot`, `sns.heatmap(df.corr())`, `sns.pairplot`, `pd.crosstab()`

TASK 3 · 20 MARKS

Feature selection

Choose the predictors, and defend the choice

20 marks

Task 3 of 5

Aim. Identify which variables carry real information about readmission, remove those that do not or that duplicate others, and justify the final feature set with statistical evidence and the insights from Task 2.

What to do

1. **Statistical tests.** For categorical predictors, use a chi-square test of independence against the target. For numeric predictors, use a t-test (or Mann–Whitney U if skewed) between readmitted and non-readmitted groups, or the point-biserial correlation. Report the statistic, the p-value and, more importantly, the size of the effect.
2. **Information-based measures.** Compute mutual information between each feature and the target to catch non-linear relationships that correlation misses.
3. **Model-based importance.** Fit a quick random forest or an L1-regularised logistic regression and report feature importances or coefficients. Consider permutation importance, which is less biased than impurity-based importance.
4. **Check redundancy.** Use the correlation matrix from Task 2 to find predictors that duplicate each other, and keep only one of each pair.
5. **Decide and justify.** Produce a table with one row per candidate feature: keep or drop, and the evidence for the decision. Consider also whether any feature is inappropriate to use (an identifier, or a sensitive attribute you cannot justify), not just whether it is predictive.

SHOW IN YOUR REPORT

- ✓ A results table: test statistic, p-value, mutual information and importance for every feature
- ✓ A chart of feature importance
- ✓ A keep/drop decision table with a justification per feature
- ✓ The final feature list used in Task 4

QUESTIONS A MARKER WILL ASK

- Did you use more than one method, and do they agree?
- Did you confuse statistical significance with practical importance?
- What did you do about correlated predictors?
- Why is Patient ID not a feature?
- Would the hospital be comfortable with every feature you kept?

Common pitfalls

- × Selecting features on the full dataset, including the test rows
- × Keeping a feature only because a p-value is small on 3,000 rows: small effects are easily "significant"
- × Dropping a feature because one method ranked it low when another ranked it high, without discussion
- × No decision table: a list of numbers is not a selection

Useful tools

`scipy.stats.chi2_contingency`, `scipy.stats.ttest_ind`, `scipy.stats.mannwhitneyu`, `scipy.stats.pointbiserialr`, `mutual_info_classif`, `SelectKBest`, `RFE`, `RandomForestClassifier.feature_importances_`, `permutation_importance`, `LogisticRegression(penalty="l1", solver="liblinear")`

TASK 4 · 20 MARKS

Model building

Compare algorithms fairly, tune them, choose one

20 marks

Task 4 of 5

Aim. Build classification models with scikit-learn, compare at least logistic regression, a decision tree and a random forest on equal terms, tune the promising ones, and select a final model for reasons you can defend to the startup.

What to do

1. **Set a baseline.** Start with a `DummyClassifier` (majority class or stratified). Every real model must beat it, and the baseline tells you what accuracy is worth.
2. **Train the candidates.** Build at least three models: logistic regression, a decision tree and a random forest. You may add others (k-nearest neighbours, gradient boosting, support vector machine) if you compare them on the same footing.
3. **Handle the class imbalance.** Use `class_weight="balanced"`, resampling (for example SMOTE from imbalanced-learn, applied inside the cross-validation folds only) or threshold adjustment, and explain your choice.
4. **Cross-validate.** Compare models with stratified k-fold cross-validation on the training set, using a threshold-free metric such as ROC-AUC (or average precision). Report the mean and the spread across folds.
5. **Tune hyperparameters.** Use `GridSearchCV` or `RandomizedSearchCV` for at least two models (for example tree depth and minimum leaf size; forest size and depth; regularisation strength C for logistic regression). Show the search space and the best settings.
6. **Select.** Choose a final model and justify the choice on performance, stability, interpretability and the hospital's needs, not on the score alone.

SHOW IN YOUR REPORT

- ✓ A table of candidate models with cross-validation scores (mean ± standard deviation)
- ✓ The hyperparameter search space and best parameters for each tuned model
- ✓ A short explanation of how each algorithm works and when it suits this problem
- ✓ A reasoned model selection

QUESTIONS A MARKER WILL ASK

- Was the comparison fair: same folds, same features, same metric?
- Did you tune on the training data only?
- Did the tuned model beat the untuned one, and by how much?
- Why this model and not the runner-up?
- How would you explain the chosen model to a clinician?

Common pitfalls

- × Comparing models on the test set and then "selecting" the winner: the test set is for Task 5 only
- × Applying SMOTE before splitting, so synthetic copies of test patients end up in training
- × A grid search with hundreds of combinations and no explanation of what the parameters do
- × Picking the random forest because "it is usually best" when the data say otherwise

Useful tools

`DummyClassifier`, `LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier`, `GradientBoostingClassifier`, `StratifiedKFold`, `cross_val_score`, `cross_validate`, `GridSearchCV`, `RandomizedSearchCV`, `validation_curve`, `Pipeline`, `imblearn.over_sampling.SMOTE`

TASK 5 · 10 MARKS

Model evaluation and testing

How good is it on patients it has never seen?

10 marks

Task 5 of 5

Aim. Evaluate the final model once on the held-out test set with metrics appropriate for an imbalanced outcome, examine overfitting and the decision threshold, and state the limitations of what you have built.

What to do

1. **Score the test set.** Report accuracy, precision, recall, F1 score and ROC-AUC for the final model (and for the other candidates, for comparison). Explain what each metric means in terms of patients: what is a false negative here, and what does it cost?
2. **Confusion matrix and curves.** Show the confusion matrix, the ROC curve and, because the classes are imbalanced, the precision–recall curve.
3. **Threshold.** The default threshold of 0.5 is rarely right. Show how precision and recall change as the threshold moves, and recommend a threshold for the hospital with a reason (for example, prioritising recall so that fewer at-risk patients are missed).
4. **Overfitting.** Compare training and test performance for each model, and show a learning curve or validation curve. Explain the strategies you used to limit overfitting: hold-out testing, cross-validation, regularisation, depth and leaf limits, ensembles, feature reduction.
5. **Limitations.** Discuss what the model cannot tell the hospital, what data would improve it, and the ethical considerations of acting on its predictions.

SHOW IN YOUR REPORT

- ✓ A metrics table for the final model (and candidates) on the test set
- ✓ Confusion matrix, ROC curve and precision–recall curve
- ✓ A train-versus-test comparison and an overfitting discussion with mitigation strategies
- ✓ A recommended threshold with a reason
- ✓ A limitations paragraph

QUESTIONS A MARKER WILL ASK

- Is the test set truly unseen: used once, at the end?
- Why is accuracy alone misleading on this data?
- Which error matters more to the hospital, and does your threshold reflect that?
- Is there a gap between training and test performance, and what did you do about it?
- What would you do next with more time or more data?

Common pitfalls

- × Reporting accuracy only, on an imbalanced outcome
- × Evaluating on the training set, or on a test set used earlier for model selection
- × Describing overfitting in theory without checking for it in your own models
- × No discussion of what the results mean for patients

Useful tools

`accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `roc_auc_score`, `classification_report`, `confusion_matrix`, `ConfusionMatrixDisplay`, `RocCurveDisplay`, `PrecisionRecallDisplay`, `precision_recall_curve`, `learning_curve`, `validation_curve`

REPORT FORMAT AND MARKING

Report format and marking

How the 100 marks are allocated

Your report must contain the sections below, in this order. *Details* is the official marking criterion; the last column describes what separates a full-mark answer from an adequate one.

CRITERIA	MARKS	DETAILS	WHAT EARNS FULL MARKS
Title page	**2**	Properly formatted title page with project title, student's name and date.	Also include the module, your student number and the dataset used.
Table of contents	**2**	Clear and well-organised table of contents.	Numbered sections with page numbers; lists of figures and tables are a bonus.
Introduction	**2**	Brief introduction of the project scope and objectives.	One page: the business problem, the objective, the dataset in one sentence, and a map of the report.
Data preprocessing	**20**	Comprehensive cleaning and preprocessing (handling missing data, normalisation, encoding).	An audit before changes; a justified rule for each change; leak-free scaling and encoding.
Exploratory data analysis	**20**	Detailed analysis with visualisations and interpretations of key variables and relationships.	Right chart for each question; every figure interpreted; insights that are used later.
Feature selection	**20**	Logical selection and justification of features based on statistical tests and data insights.	More than one method; effect sizes, not only p-values; a keep/drop decision per feature.
Model development	**20**	Application of appropriate machine learning models, including model tuning and optimisation.	A baseline; three or more algorithms on equal footing; cross-validated tuning; a reasoned selection.
Model evaluation	**10**	Thorough evaluation using appropriate metrics (e.g. accuracy, ROC-AUC). Discussion of model performance and limitations.	Full metric set on a true hold-out; threshold reasoning; overfitting checked, not just described.
Conclusion	**2**	A concise summary of findings, implications and potential improvements.	Answers the main question directly; recommendations for the hospital; next steps.
List of references	**2**	Correctly formatted references and adequate sourcing of external data and literature.	Consistent Harvard style; cite the libraries, the literature and any tutorials or AI tools used.
Total	**100**		

SUBMISSION

Submission, workflow and integrity

What to hand in, how to pace the work, and the rules

What to submit

ITEM	FORMAT	REQUIREMENTS
Report	PDF	Follows the structure on the previous page. File name: `Surname_Initials_Readmission_Report.pdf`.
Notebook	.ipynb (or .py)	All code, runnable from top to bottom on the supplied CSV with outputs shown. Clear section headings matching the five tasks. File name: `Surname_Initials_Readmission_Notebook.ipynb`.
Declaration	In the report	A short statement that the work is your own, listing any sources, collaborators and AI tools used and for what.

Submit both files through the platform and by the deadline given by your facilitator. The dataset itself does not need to be submitted.

A suggested three-week workflow

WEEK	FOCUS	MILESTONES
Week 1	Tasks 1 and 2	Data loaded and audited; cleaning rules decided and coded; first set of EDA charts with interpretations; insights list written.
Week 2	Tasks 3 and 4	Statistical tests and importance measures done; feature decision table complete; baseline and three candidate models cross-validated; tuning finished; model selected.
Week 3	Task 5 and the write-up	Test-set evaluation, threshold analysis and overfitting check; report drafted, figures captioned, references compiled; checklist in Appendix B completed; final proofread.

Academic integrity and the use of AI tools

This is an individual assessment. You may discuss ideas with classmates, but the code, analysis and writing you submit must be your own. Generative AI tools may be used in the way a textbook or documentation is used: to understand a concept, to look up a function or to debug an error. Any such use must be declared in your report, including which tool and for what. Submitting code or text produced by a tool without understanding it, or without declaring it, is academic misconduct. You may be asked to explain any part of your submission in person.

> **Getting help**
>
> If you are stuck, bring the specific problem and what you have tried to your facilitator or the course forum. Questions about the brief itself (for example, what a field means) should be asked early so that everyone receives the same answer.

Appendix A · Python quick reference

The functions and classes you are most likely to need, by task. The scikit-learn user guide (https://scikit-learn.org/stable/user_guide.html) documents every one of them with examples.

TASK	PURPOSE	FUNCTION OR CLASS	LIBRARY
1	Inspect structure and types	`df.info()`, `df.describe(include="all")`	pandas
1	Find missing values and duplicates	`df.isna().sum()`, `df.duplicated().sum()`	pandas
1	Clean text labels	`df[c].str.strip().str.title()`, `df[c].map(...)`	pandas
1	Impute	`SimpleImputer(strategy="median")`, `KNNImputer`	sklearn.impute
1	Encode	`pd.get_dummies(drop_first=True)`, `OneHotEncoder`	pandas / sklearn.preprocessing
1	Scale	`StandardScaler`, `MinMaxScaler`	sklearn.preprocessing
1	Combine steps without leakage	`ColumnTransformer`, `Pipeline`	sklearn.compose / sklearn.pipeline
1	Split	`train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)`	sklearn.model_selection
2	Distributions	`sns.histplot`, `sns.boxplot`, `sns.violinplot`	seaborn
2	Rates by group	`df.groupby(c)[t].mean()`, `sns.barplot`, `pd.crosstab(normalize="index")`	pandas / seaborn
2	Correlation	`df.corr(numeric_only=True)`, `sns.heatmap(annot=True)`	pandas / seaborn
2	Figure layout	`plt.subplots(nrows, ncols, figsize=...)`, `fig.tight_layout()`	matplotlib
3	Categorical association	`chi2_contingency(pd.crosstab(a, b))`	scipy.stats
3	Numeric group difference	`ttest_ind(a, b, equal_var=False)`, `mannwhitneyu`, `pointbiserialr`	scipy.stats
3	Information and importance	`mutual_info_classif`, `permutation_importance`, `RFE`	sklearn
4	Baseline	`DummyClassifier(strategy="most_frequent")`	sklearn.dummy
4	Models	`LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier`	sklearn
4	Cross-validation and tuning	`StratifiedKFold`, `cross_val_score`, `GridSearchCV`, `RandomizedSearchCV`	sklearn.model_selection
4	Imbalance	`class_weight="balanced"`, `SMOTE` (inside a `imblearn.pipeline.Pipeline`)	sklearn / imbalanced-learn
5	Metrics	`classification_report`, `roc_auc_score`, `confusion_matrix`	sklearn.metrics

TASK	PURPOSE	FUNCTION OR CLASS	LIBRARY
5	Curves	`RocCurveDisplay.from_estimator`, `PrecisionRecallDisplay.from_estimator`	sklearn.metrics
5	Overfitting	`learning_curve`, `validation_curve`	sklearn.model_selection

A leak-free skeleton to start from

PYTHON · EVERYTHING THAT LEARNS FROM THE DATA IS FITTED INSIDE THE PIPELINE, ON TRAINING ROWS ONLY

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, cross_val_score

numeric = ["Age", "Length of Stay", "Number of Diagnoses", "Blood Pressure",
           "Blood Sugar Levels", "Previous Admissions"]
categorical = ["Gender", "Admission Type"]

X = df[numeric + categorical]
y = (df["Readmission"] == "Yes").astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")), ("scale", StandardScaler())]), numeric),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(drop="first", handle_unknown="ignore"))]), categorical),
])
model = Pipeline([("prep", preprocess),
                  ("clf", LogisticRegression(max_iter=1000, class_weight="balanced"))])

scores = cross_val_score(model, X_train, y_train, cv=5, scoring="roc_auc")
print(f"CV ROC-AUC: {scores.mean():.3f} ± {scores.std():.3f}")
```

Appendix B · Report template and checklist

Report template

SECTION	SUGGESTED LENGTH	CONTENTS
Title page	1 page	Project title, your name and student number, module, date, dataset name
Table of contents	1 page	Numbered headings with page numbers
1 Introduction	½ – 1 page	Business problem, objective, the dataset in a sentence, structure of the report
2 Data preprocessing	2 – 3 pages	Audit, cleaning rules and justification, encoding, scaling, split
3 Exploratory data analysis	3 – 4 pages	Target balance, distributions, relationships, correlation, insights
4 Feature selection	2 – 3 pages	Methods, results table, importance chart, decision table, final features
5 Model development	2 – 3 pages	Baseline, candidates, cross-validation, tuning, selection
6 Model evaluation	2 pages	Test metrics, confusion matrix, curves, threshold, overfitting, limitations
7 Conclusion	½ – 1 page	Answer to the main question, implications for the hospital, improvements
References	½ page	Harvard style, alphabetical
Appendices	As needed	Full code listing or notebook link, additional figures, declaration

Presentation standards

- Number every figure and table and give each a caption that states what it shows
- Put long code listings in an appendix; keep short, essential snippets in the body
- Use consistent units and decimal places (three decimals for metrics such as 0.742)
- Write in the past tense for what you did and the present tense for what the results show
- Cite every external source, including documentation, tutorials and generative AI tools, in Harvard style
- Make sure every number in the text can be traced to an output in your notebook
- Proofread: a report with inconsistent figures loses credibility quickly
- Aim for a report of 3,000 to 5,000 words excluding code, figures and appendices (guideline, not a limit)

Self-assessment checklist

Tick each item before you submit. Markers use a very similar list.

- ☐ I audited the data before changing it and reported what I found
- ☐ Every cleaning, encoding and scaling decision has a written reason
- ☐ The train/test split happens before any scaler, imputer or model is fitted

- [ ] Every figure has a title, labelled axes with units, a number and a caption
- [ ] Every figure and table is interpreted in the text
- [ ] I noted the class balance and chose metrics that suit it
- [ ] Feature selection uses at least two methods and ends in a keep/drop table
- [ ] I compared a baseline and at least three algorithms on the same cross-validation folds
- [ ] Hyperparameters were tuned on training data only
- [ ] The test set was used once, at the end, for the final numbers
- [ ] I report accuracy, precision, recall, F1 and ROC-AUC, plus a confusion matrix and curves
- [ ] I discussed the decision threshold and recommended one
- [ ] I checked for overfitting in my own models and explained how I limited it
- [ ] The conclusion answers the main question and gives the hospital a recommendation
- [ ] References are complete and in Harvard style, including libraries and AI tools
- [ ] The notebook runs from top to bottom without errors and its numbers match the report

References used in this brief

1. Kansagara, D., Englander, H., Salanitro, A., Kagen, D., Theobald, C., Freeman, M. and Kripalani, S. (2011) 'Risk prediction models for hospital readmission: a systematic review', *JAMA*, 306(15), pp. 1688–1698.
2. Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
3. van Walraven, C., Dhalla, I.A., Bell, C., Etchells, E., Stiell, I.G., Zarnke, K., Austin, P.C. and Forster, A.J. (2010) 'Derivation and validation of an index to predict early death or unplanned readmission after discharge from hospital to the community', *CMAJ*, 182(6), pp. 551–557.